

Unit 1 - Math Fundamentals

Floating-Point Arithmetic

MSDS 520

Meharry Medical College

Outline

Why Finite Precision?

A Problem to Motivate Us

Type this into Python

```
>>> 0.1 + 0.2 == 0.3  
False
```

Not a bug. Not a Python quirk, the same thing happens in R, C, Java and Excel. Ask what the sum actually *is* and the machine tells you:

```
>>> 0.1 + 0.2  
0.30000000000000004
```

Something has gone wrong four digits past where anyone was looking, and an `if` statement that compares two numbers for equality will act on it. This lecture is about **where that error comes from, how big it can get, and how to write code that survives it.**

Where This Shows Up in Data Science

Silent, not loud

- **Filtering a dataframe** on `df[col] == 0.3` quietly returns no rows.
- **A convergence check** of “stop when the error is 0” never fires, and the loop runs forever.
- **A variance computed the naive way** comes out *negative* on data with a large mean.
- **Multiplying many probabilities** together collapses to exactly 0, and the log of that is $-\infty$.
- **Adding a million small numbers** gives a different answer depending on the order you add them.

None of these raise an error. They give you a wrong number and let you publish it. Recognizing the symptoms is a genuinely practical skill.

The Root of the Problem

Unit 1-2 built the real numbers $\mathbb{R}$, uncountably many of them, most requiring infinitely many digits:

$$\pi = 3.14159265358979323846\dots \quad \frac{1}{3} = 0.333\dots \quad \sqrt{2} = 1.41421356\dots$$

A computer has finite memory

A number is stored in a fixed number of **bits**, 64 of them, typically. With 64 bits there are only 2^{64} possible patterns, so a machine can represent only **finitely many** numbers exactly.

Every other real number must be replaced by the nearest one the machine *does* have. That substitution is called **rounding**, and the difference it creates is **rounding error**.

This is not a defect to be fixed. It is a permanent, unavoidable consequence of finite memory, and every numerical result you ever compute carries some of it.

What We Will Build

The plan

- ① **Binary**: how a machine writes numbers, and how to convert both ways.
- ② The **floating-point number system**, scientific notation in base 2.
- ③ Why 0.1 **is not exact**, and what to do about it.
- ④ **Machine epsilon** and relative error.
- ⑤ The four classic failures: **absorption, cancellation, overflow, underflow**.
- ⑥ Practical rules for writing numerical code that behaves.

Notebook for this lecture: U1-5.FloatingPoint-1_MachineEpsilon.

Numbers in Base 2

What a Base Actually Means

The digits 253 are shorthand for a sum of powers of ten, the position of a digit tells you which power it multiplies:

$$253_{10} = 2 \times 10^2 + 5 \times 10^1 + 3 \times 10^0$$

Nothing about that is special to ten. Use 2 as the base and the same scheme works with only the digits 0 and 1:

$$11001_2 = 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 16 + 8 + 1 = 25_{10}$$

Why base 2 for machines, base 10 for us

A transistor is reliably **off** or **on**, which is exactly two states, so two digits is what the hardware naturally offers. Distinguishing ten voltage levels would be far harder to build and far easier to get wrong. Base 10 is an accident of having ten fingers, not a mathematical preference.

The subscript matters: 11001 means twenty-five in binary and eleven thousand and one in decimal. Always say which base you mean.

Binary to Decimal: Just Add Up the Places

Integer part

Positions count from 0, right to left:

$$\begin{aligned}11001_2 &= 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^0 \\&= 16 + 8 + 1 \\&= 25_{10}\end{aligned}$$

Fractional part

To the right of the point, exponents go *negative*:

$$\begin{aligned}0.101_2 &= 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} \\&= \frac{1}{2} + \frac{1}{8} \\&= 0.625_{10}\end{aligned}$$

Put the two halves together and you have the whole number:

$$11001.101_2 = 25.625_{10}$$

Note the shape of the available fractions: halves, quarters, eighths, sixteenths. **A binary fraction is a sum of powers of one half**, and nothing else is available. That single fact explains everything in section 4.

Decimal to Binary: The Integer Part

Repeatedly divide by 2 and record the remainders, then read them *bottom to top*.

Division	Remainder
$25 \div 2 = 12$	1
$12 \div 2 = 6$	0
$6 \div 2 = 3$	0
$3 \div 2 = 1$	1
$1 \div 2 = 0$	1

Reading the remainder column from the bottom up:

$$25_{10} = 11001_2$$

Why it works: each division by 2 strips off the last binary digit, exactly as dividing by 10 strips off the last decimal digit. The remainder *is* that digit.

The process always stops, the quotient shrinks every step and must reach 0.

Check it: $16 + 8 + 1 = 25$. ✓ Converting back is always the cheapest way to catch an arithmetic slip.

Decimal to Binary: The Fractional Part

Repeatedly multiply by 2 and record the integer parts, then read them *top to bottom*.

Operation	Integer part
$0.625 \times 2 = 1.25$	1
$0.25 \times 2 = 0.5$	0
$0.5 \times 2 = 1.0$	1

Reading the integer column from the top down:

$$0.625_{10} = 0.101_2$$

After each step, carry only the **fractional** part forward and multiply again. Stop when the fractional part hits zero.

And if it never hits zero?

Then the number has no finite binary representation, and the digits repeat forever. **That is the normal case, not the exception**, and it happens to 0.1, as we will see shortly.

Combining halves: $25.625_{10} = 11001.101_2$, convert the two sides of the point separately, then join them.

Exercise: Converting Bases

Try it

- ① Convert 19_{10} to binary.
- ② Convert 0.375_{10} to binary.
- ③ Convert 1101.01_2 to decimal.
- ④ Try converting 0.2_{10} to binary. What happens, and why?

Exercise: Solution

1 to 3

(1) Remainders of 19, 9, 4, 2, 1 divided by 2 are 1, 1, 0, 0, 1; read bottom-up, $19_{10} = 10011_2$. Check: $16 + 2 + 1 = 19$. ✓

(2) $0.375 \times 2 = 0.75 \rightarrow 0$; $0.75 \times 2 = 1.5 \rightarrow 1$; $0.5 \times 2 = 1.0 \rightarrow 1$. So $0.375_{10} = 0.011_2$. Check: $\frac{1}{4} + \frac{1}{8} = 0.375$. ✓

(3) $1101.01_2 = 8 + 4 + 1 + \frac{1}{4} = 13.25_{10}$.

4. The one that never ends

$0.2 \times 2 = 0.4 \rightarrow 0$; $0.4 \times 2 = 0.8 \rightarrow 0$; $0.8 \times 2 = 1.6 \rightarrow 1$; $0.6 \times 2 = 1.2 \rightarrow 1$; $0.2 \times 2 = \dots$, we are back where we started, so the block 0011 repeats forever: $0.2_{10} = 0.\overline{0011}_2$.

The reason is the same one that makes $\frac{1}{3}$ repeat in decimal: $0.2 = \frac{1}{5}$, and 5 is not a power of 2, so no finite sum of halves, quarters and eighths can ever land on it exactly.

The Floating-Point Number System

Scientific Notation, in Any Base

Scientists write a wide-ranging number as a modest number times a power of ten:

$$25.625 = \underbrace{+}_{\text{sign}} \underbrace{2.5625}_{\text{mantissa}} \times \underbrace{10^1}_{\substack{\text{exponent} \\ \text{base}}}$$

A computer does exactly the same thing in base 2:

$$11001.101_2 = + 1.1001101_2 \times 2^4$$

Why “floating” point

The binary point does not sit in a fixed place, the exponent **floats** it left or right. That is what lets 64 bits cover everything from the mass of a proton to the mass of a galaxy, instead of a narrow fixed range.

The trade is the theme of this lecture: enormous **range** bought with a fixed number of **significant digits**. You always get about the same number of digits, never more, no matter the magnitude.

The Formal Definition

A floating-point system

Four integers define one: base β , precision p , and an exponent range $[L, U]$. Every representable number has the form

$$x = \pm \left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}} \right) \beta^E$$

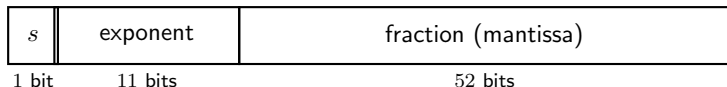
with digits $0 \leq d_i \leq \beta - 1$ and exponent $L \leq E \leq U$.

- The d_i are the digits of the **mantissa**, there are exactly p of them, and that is the **precision**.
- Most systems are **normalized**: $d_0 \neq 0$. In base 2 that forces $d_0 = 1$, so the leading bit is known in advance and need not be stored, a free extra digit of precision.

Normalization also makes the representation **unique**: without it, 0.1×2^5 and 1.0×2^4 would be two spellings of the same value.

What 64 Bits Buy You

The IEEE 754 **double**, the default float in Python and NumPy:



- **Precision:** 52 stored bits plus the free leading 1 gives $p = 53$, about 16 decimal digits, since $2^{53} \approx 9 \times 10^{15}$.
- **Range:** the 11 exponent bits give roughly 10^{-308} to 10^{308} .
- **Count:** finitely many values, a little under 2^{64} of them, scattered across that entire range.

Sixteen digits sounds like plenty, and usually is. The failures later in this lecture are all cases where an algorithm *destroys* most of those sixteen digits in a single subtraction.

Three Numbers Worth Naming

The edges of the system

- **UFL** (underflow limit), the smallest positive normalized number:

$$\text{UFL} = \beta^L$$

- **OFL** (overflow limit), the largest representable number:

$$\text{OFL} = \beta^{U+1} (1 - \beta^{-p})$$

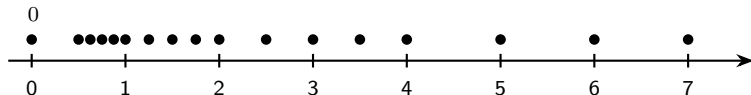
- **Total count** of normalized values:

$$2 (\beta - 1) \beta^{p-1} (U - L + 1) + 1$$

Read the count left to right: two signs, $(\beta - 1)$ leading digits, β^{p-1} remaining mantissas, $(U - L + 1)$ exponents, plus 1 for zero, which normalized form cannot otherwise express. Ask for more than OFL and you get ∞ ; ask for less than UFL and you get 0, both silently.

A Toy System You Can Draw

Take $\beta = 2$, $p = 3$, $L = -1$, $U = 2$. Every positive value is $(1.d_1d_2)_2 \times 2^E$, four mantissas $\{1, 1.25, 1.5, 1.75\}$ times four exponents:



The spacing is the point

The gaps are 0.125 between 0.5 and 1, then 0.25, then 0.5, then 1, **doubling with every exponent**. Floating-point numbers are dense near zero and sparse out at the extremes.

So the *absolute* error of rounding grows with magnitude, while the *relative* error stays roughly constant. Notice too the wide empty gap between 0 and UFL = 0.5, there is nothing at all in there.

Special Values

Some bit patterns are reserved for things that are not ordinary numbers:

Value	Produced by	Behaves like
inf	1/0, overflow past OFL	absorbs: $\infty + x = \infty$
-inf	-1/0, $\ln(0)$	likewise
nan	0/0, $\infty - \infty$, $\sqrt{-1}$	poisons: $\text{nan} + x = \text{nan}$
-0.0	underflow of a negative	equals 0.0 when compared

The one rule everybody trips over

nan is not equal to anything, **including itself**. `nan == nan` is False.

Test with `math.isnan(x)` or `np.isnan(x)`, never with `==`. And since nan propagates through every operation it touches, one bad value early in a pipeline can turn a whole column into nan downstream.

Exercise: A Toy Floating-Point System

Try it

Work in the system $\beta = 2$, $p = 3$, $L = -1$, $U = 2$ from the number line.

- ① What is UFL? What is OFL?
- ② How many numbers does this system contain in total?
- ③ What is the gap between 1 and the next representable number above it? Between 4 and the next one above *that*?
- ④ Is 1.3 representable? If not: what does it round to?

Exercise: Solution

1 and 2

(1) $UFL = \beta^L = 2^{-1} = 0.5$. $OFL = \beta^{U+1}(1 - \beta^{-p}) = 2^3(1 - \frac{1}{8}) = 8 \cdot \frac{7}{8} = 7$. Both match the ends of the number line.

(2) $2(\beta - 1)\beta^{p-1}(U - L + 1) + 1 = 2 \cdot 1 \cdot 4 \cdot 4 + 1 = 33$: sixteen positive values, sixteen negative, and zero.

3 and 4

(3) Above 1 the next value is 1.25, a gap of $0.25 = \beta^{1-p}$. Above 4 the next is 5, a gap of 1, four times larger, because the exponent has gone up by two.

(4) No. It falls between 1.25 and 1.5 and rounds to the nearer, 1.25, an absolute error of 0.05. **A number being “simple” in decimal says nothing about whether the machine can hold it.**

Why 0.1 Is Not Exact

The Same Problem You Already Accept in Decimal

Nobody is disturbed that $\frac{1}{3}$ has no finite decimal form:

$$\frac{1}{3} = 0.3333\dots$$

Write it with ten digits and you have stored something slightly *smaller* than a third. The base simply cannot express it in finitely many digits, because 3 does not divide any power of 10.

Binary has its own version of that list, and it is longer

A fraction terminates in base 2 exactly when its denominator (in lowest terms) is a **power of 2**.

So $\frac{1}{2}$, $\frac{1}{4}$, $\frac{3}{8}$, 0.625 are all exact, while $\frac{1}{10}$, $\frac{1}{5}$, $\frac{1}{3}$ are not. Tenths are exact in base 10 and *never* exact in base 2.

And tenths are exactly what human data is full of: prices, doses, measurements, thresholds. **The numbers people write down are the ones binary handles worst.**

What the Machine Actually Stores for 0.1

Run the fractional conversion on 0.1 and it cycles forever:

$$0.1_{10} = 0.0\overline{0011}_2 = 0.000110011001100\dots_2$$

The machine keeps 53 significant bits and rounds the rest away. Printed to 20 decimal places, the stored value is:

```
>>> format(0.1, '.20f')  
0.100000000000000000555
```

So the errors were there before you did any arithmetic

0.1, 0.2 and 0.3 are each stored slightly off. Add the first two and the two small errors combine into one big enough to see:

$$0.1 + 0.2 \rightarrow 0.300000000000000004 \neq 0.3$$

Python prints 0.1 as “0.1” because it shows the shortest decimal that reads back to the same bits, a convenience that hides the discrepancy until an arithmetic operation exposes it.

Never Compare Floats with ==

The rule

Two floating-point numbers that *should* be equal mathematically are usually not equal bit-for-bit. Ask instead whether they are **close enough**:

$$|a - b| < \text{tol}$$

Broken

```
if x == 0.3:  
if err == 0:  
df[df.price == 0.1]
```

`np.isclose` and `math.isclose` scale the tolerance with the size of the numbers, which is what you usually want: being within 10^{-9} is a strict demand near 1 and an absurdly loose one near 10^{12} .

Two exceptions, worth knowing: **integers** stored as floats are exact up to 2^{53} , and comparing a float to **exactly zero** that you assigned yourself is fine.

Anything that came out of arithmetic gets a tolerance.

Correct

```
if abs(x - 0.3) < 1e-9:  
if err < 1e-12:  
np.isclose(a, b)
```

Exercise: Exact or Not?

Try it

- ① Which of these are stored exactly in binary floating point?
0.5, 0.75, 0.1, 0.3, 2.25, $\frac{1}{3}$
- ② Explain in one sentence why $0.5 + 0.25 == 0.75$ is True while $0.1 + 0.2 == 0.3$ is False.
- ③ A colleague filters a price column with `df[df.price == 19.99]` and gets zero rows, though the value is plainly in the data. What happened, and what should they write instead?

Exercise: Solution

1 and 2

(1) Exact: $0.5 = \frac{1}{2}$, $0.75 = \frac{3}{4}$, $2.25 = \frac{9}{4}$, all have a power of 2 underneath. Not exact: $0.1 = \frac{1}{10}$, $0.3 = \frac{3}{10}$, $\frac{1}{3}$.

(2) Halves and quarters are exactly representable, so that sum is exact arithmetic on exact inputs; tenths are not representable at all, so the second comparison is asking whether two different roundings of different quantities happen to land on identical bits, and they do not.

3. The price column

The stored value is the nearest double to 19.99, and the literal 19.99 in their code may round from a different computation, so the bits need not match.

Use a tolerance, `df[np.isclose(df.price, 19.99)]`, or better, **store money in integer cents** and compare exactly. Currency is a case where floating point is the wrong tool entirely.

Rounding and Machine Epsilon

The Rounding Function $\text{fl}(x)$

Definition

Write $\text{fl}(x)$ for the floating-point number nearest to the real number x . Storing a number means replacing x by $\text{fl}(x)$, and the difference $\text{fl}(x) - x$ is the **rounding error**.

Every arithmetic operation rounds too. What the machine computes is not $a + b$ but

$$\text{fl}(\text{fl}(a) + \text{fl}(b))$$

, three chances to round in a single addition. Those errors accumulate over millions of operations, which is why a long computation deserves more suspicion than a short one.

Rounding is to the *nearest* representable value, with ties going to the one whose last bit is even. That last detail keeps ties from biasing systematically upward, which is what would happen if they always rounded away from zero.

Rounding to an Integer: Three Different Functions

Name	Meaning	$2.5 \mapsto$	$-2.5 \mapsto$
$\lfloor x \rfloor$ floor	largest integer $\leq x$	2	-3
$\lceil x \rceil$ ceiling	smallest integer $\geq x$	3	-2
round	nearest integer	2	-2

Watch the negatives, and watch the ties

Floor is **not** “chop off the decimals”, for negative numbers it moves *away* from zero.

And Python’s round breaks ties toward the **even** number, so `round(0.5)` is 0 while `round(1.5)` is 2. This is deliberate (it removes upward bias when rounding many values), and it surprises everyone the first time.

Integer division `//` in Python is floor division, so `-7 // 2` is -4, not -3.

Machine Epsilon

Definition

Machine epsilon ε is the smallest number for which the machine can still tell $1 + \varepsilon$ apart from 1:

$$\text{fl}(1 + \varepsilon) > 1$$

Equivalently, it is the **gap** between 1 and the next representable number above it: $\varepsilon = \beta^{1-p}$.

- For the toy system: $\varepsilon = 2^{1-3} = 0.25$, exactly the gap from 1 to 1.25 on the number line.
- For a double: $\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$, about 16 decimal digits.

In Python it is `sys.float_info.epsilon`, or `np.finfo(float).eps`. **Anything smaller than ε added to 1 vanishes without a trace**, and, scaled to any magnitude, that is the single most useful fact in this lecture.

Think in Relative Error, Not Absolute

Rounding to the nearest representable value costs at most half a gap, and the gaps scale with magnitude. That gives the fundamental bound:

$$\frac{|\text{fl}(x) - x|}{|x|} \leq \frac{\varepsilon}{2}$$

What that means in practice

You get about 16 **significant digits**, wherever you are on the number line, not 16 decimal places. Near 10^{-8} the absolute error is minuscule; near 10^8 the same relative error is an absolute error of about 10^{-8} .

So “is this error big?” is not answerable without knowing the scale. An error of 0.001 is catastrophic in a probability and irrelevant in a population count. **Always ask about error relative to the size of the thing you are computing**, and pick tolerances the same way.

Exercise: Epsilon

Try it

- ① Predict the output of $1.0 + 1e-20 == 1.0$. Why?
- ② Predict the output of $1e20 + 1.0 == 1e20$. Why?
- ③ If a double holds about 16 significant digits, roughly how large is the gap between representable numbers near 10^6 ?
- ④ Machine epsilon is about 10^{-16} , yet doubles can represent numbers as small as 10^{-308} . Is that a contradiction?

Exercise: Solution

1 and 2

(1) True. 10^{-20} is far below $\varepsilon \approx 2.2 \times 10^{-16}$, so the exact sum lies nearer to 1 than to the next representable value, and rounds back to 1. The addition accomplishes nothing.

(2) Also True, and for the same reason scaled up: near 10^{20} the gap between neighbors is about $10^{20} \cdot \varepsilon \approx 2 \times 10^4$, so adding 1 cannot move you to the next value.

3 and 4

(3) The gap is roughly $|x| \cdot \varepsilon \approx 10^6 \times 2.2 \times 10^{-16} \approx 2 \times 10^{-10}$. Big numbers have coarse neighborhoods.

(4) No, the two describe different things. ε is about **precision** (how many digits), UFL and OFL are about **range** (which magnitudes). 10^{-308} is a perfectly fine *value*; it simply still carries only 16 significant digits, and it still disappears if you add it to 1.

When Arithmetic Misbehaves

Addition Is Not Associative

Mathematically $(a + b) + c = a + (b + c)$. On a machine, that fails:

```
>>> a, b, c = 1e16, 1.0, 1.0
>>> (a + b) + c                                1e+16
>>> a + (b + c)                                1.0000000000000002e+16
```

Why the two orders differ

Near 10^{16} the gap between representable numbers is 2. Grouped on the left, each 1 is absorbed separately and neither one is large enough to move the total. Grouped on the right, $b + c$ makes a 2 first, and a 2 is exactly one gap, so it survives.

The lesson generalizes: when summing many values, **add the small ones together first** so they can accumulate into something big enough to register. `math.fsum` and `np.sum` use smarter strategies than a plain loop for exactly this reason.

Cancellation: Subtracting Two Close Numbers

Subtraction of nearly equal numbers is the most destructive operation in numerical computing. Both inputs are good to 16 digits, and the answer is not:

$$\begin{array}{rcl} x & = & 1.000000100000000000 \quad (16 \text{ good digits}) \\ y & = & 1.000000000000000000 \quad (16 \text{ good digits}) \\ \hline x - y & = & 0.000000100000000000 \quad (\text{only } 9 \text{ good digits}) \end{array}$$

Where the digits went

The leading digits were identical, so they cancelled to zero, and cancelling carries no information. What is left is the tail, where the rounding error already lived. The error did not grow; the **result** shrank, so the error became a much larger *share* of it.

This is called **catastrophic cancellation**, and it is a property of the *formula*, not of the data. Rearranging the algebra usually avoids it.

Cancellation in the Wild: Variance

The textbook shortcut for variance is a subtraction of two nearly equal numbers:

$$s^2 = \underbrace{\frac{1}{n} \sum_{i=1}^n x_i^2 - \bar{x}^2}_{\text{one pass, numerically awful}}$$

$$s^2 = \underbrace{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}_{\text{two passes, well behaved}}$$

Try it on lab values near 1000 with a spread of 0.01

Then $\frac{1}{n} \sum x_i^2 \approx 10^6$ and $\bar{x}^2 \approx 10^6$ agree to about ten digits, and the difference, the answer, keeps only the last few. The result can come out as garbage, or even **negative**, which is impossible for a sum of squares.

The right-hand form subtracts *first*, while the numbers are still small and their difference is meaningful, and only then squares. Same mathematics, completely different behavior in finite precision. **This is why you use `np.var` rather than writing the shortcut yourself.**

Overflow and Underflow

Overflow: past OFL

```
>>> np.exp(1000)
inf
```

About 10^{434} , past the largest double. Everything computed from it is now `inf` or `nan`.

Underflow: below UFL

```
>>> np.exp(-1000)
0.0
```

A tiny positive number becomes exactly zero, and then $\ln(0) = -\infty$, or a division by it explodes.

The classic fix: work in logs

Multiplying 1000 probabilities underflows to 0. Taking logarithms turns the product into a sum (Unit 1-4) and the magnitudes back into something a double can hold:

$$\prod_{i=1}^n p_i \longrightarrow \sum_{i=1}^n \ln p_i$$

That is why statistics maximizes the **log**-likelihood, and why classifiers expose `predict_log_proba`, a numerical decision, not a mathematical one.

Exercise: Diagnosing the Failure

Try it

Name which failure mode each one is, and say how to fix it.

- ① A loop `while error != 0`: never terminates.
- ② Summing a column of a million small measurements gives a different total in NumPy than in a hand-written Python loop.
- ③ A computed variance comes out as -3.2×10^{-6} .
- ④ A likelihood computed as a product of 500 probabilities prints as 0.0, and its logarithm as `-inf`.

Exercise: Solution

1 and 2

(1) Testing a float for exact equality. The error shrinks toward zero but never lands on it exactly. Use a tolerance: `while error > 1e-12:`

(2) Absorption plus non-associativity, the order of the additions differs, so the absorbed terms differ. NumPy sums in blocks rather than strictly left to right, which loses less. Neither is “wrong”; use `math.fsum` when you need the exact total.

3 and 4

(3) Catastrophic cancellation in the $\frac{1}{n} \sum x_i^2 - \bar{x}^2$ shortcut. A sum of squares cannot be negative, so this is arithmetic error made visible. Subtract the mean first, or just call `np.var`.

(4) Underflow: the product fell below UFL and became exactly 0. Sum the logs of the probabilities instead of multiplying them, and never take the log of a product you computed numerically.

Rules of Thumb

Writing Code That Survives

Six habits

- ① **Never compare floats with `==`.** Use `np.isclose` or an explicit tolerance, chosen relative to the scale of the numbers.
- ② **Avoid subtracting nearly equal numbers.** If a formula demands it, look for an algebraic rearrangement that does not.
- ③ **Add small numbers before large ones:** and prefer `math.fsum` or `np.sum` to a hand-rolled loop.
- ④ **Work in logs** whenever many probabilities or very large or small factors multiply together.
- ⑤ **Use the library version.** `np.var`, `np.linalg.solve` and `scipy.special.logsumexp` already contain these fixes.
- ⑥ **Use integers where the data is exact:** money in cents, counts as counts. Floating point is for measurements, not for ledgers.

Keeping This in Proportion

Floating point is not broken

Sixteen significant digits is more precision than nearly any measurement deserves. A blood pressure reading is good to three digits; a lab assay, maybe five. The arithmetic is not your accuracy bottleneck, **the data is**.

The failures in this lecture are not really about tiny errors. Each one is a case where an **algorithm amplifies** a tiny error into a large one:

- cancellation turns a 10^{-16} relative error into a 10^{-7} one,
- absorption discards a term completely,
- underflow discards an entire probability.

So the skill worth taking away is not fear of decimals. It is recognizing the **shapes**, a difference of near-equal quantities, a sum spanning many magnitudes, a long product, and reaching for the stable formulation before the numbers get a chance to misbehave.

Review

Review

What we covered

- ① Memory is finite, so only finitely many reals are representable; every other one is **rounded** to a neighbor.
- ② **Binary** places are powers of 2, divide by 2 for the integer part, multiply by 2 for the fraction.
- ③ A **floating-point system** is (β, p, L, U) : sign, mantissa, exponent. Doubles give $p = 53$ bits, about 16 digits, and a range of roughly $10^{\pm 308}$.
- ④ Spacing **doubles with each exponent**: absolute error grows with magnitude, relative error does not. **UFL/OFL** are the edges; past them live `inf` and `nan`, and `nan != nan`.
- ⑤ A fraction is exact in binary only if its denominator is a power of 2, so 0.1 never is: **compare floats with a tolerance**.
- ⑥ **Machine epsilon** $\epsilon = 2^{-52}$ is the gap above 1; relative rounding error is at most $\epsilon/2$.
- ⑦ Four failure modes: **absorption, cancellation, overflow, underflow**, all fixable by rearranging the computation, not by adding digits.

Notebook: U1-5_FloatingPoint-1_MachineEpsilon. **Previous:** Unit 1-4, Series.

Next: Unit 2, **Linear Algebra**.